



Lab Assignment 3

Course: Higher Level Programming Language Lab

Course Code: CSEL-3204

Submitted To

Dr. MD. Manowarul Islam

Professor

Department of CSE

Jagannath University

Submitted By

Maisha Binte Monir

ID: B210305016

Session: 2021-22 (CSE-13)

Task 1: Line Plot

Create a line chart showing monthly sales. Add title, labels, grid, markers, and explain the sales trend. Sample example

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
```

```
sales = [120, 150, 170, 160, 200, 220]
```

```
import matplotlib.pyplot as plt
```

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug"]
```

```
sales = [120, 135, 150, 145, 170, 190, 200, 180]
```

```
plt.plot(months, sales, marker='o')
```

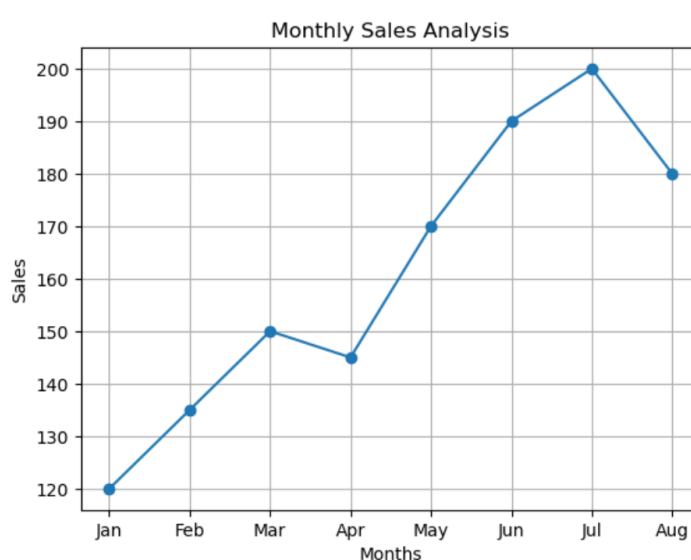
```
plt.title("Monthly Sales Analysis")
```

```
plt.xlabel("Months")
```

```
plt.ylabel("Sales")
```

```
plt.grid(True)
```

```
plt.show()
```



Output:

From the dataset: Jan -> Mar: steady increase (120 -> 150)

Apr: slight dip (150 -> 145), possible seasonal fluctuation or demand slowdown

May -> Jul: strong growth (145 -> 190), indicating recovery and expansion

Jul -> Aug: Sharp decline (200 -> 180), indicating dramatic sales decrease

Overall: General upward growth trend Minor temporary decline in April Strong positive momentum in later months Sharp decline in August

Task 2: Multiple Line Plot

Compare two students' performances across subjects using different line styles and a legend. Interpret the results.

```
subjects = ["Math", "Physics", "Chemistry", "Biology", "English", "ICT"]
```

```
student_A = [85, 78, 90, 88, 76, 92]
```

```
student_B = [80, 82, 87, 85, 79, 89]
```

Requirements:

- Use different line styles and markers.
- Add a legend.
- Add a suitable title.

```
import numpy as np
```

```
subjects = ["Math", "Physics", "CSE", "English", "History"]
```

```
student1 = [78, 85, 90, 87, 78]
```

```
student2 = [65, 70, 72, 77, 78]
```

```
x = np.arange(len(subjects))
```

```
width = 0.35
```

```
plt.plot(x - width/2, student1, linestyle='-', marker='o',  
label="Student 1")
```

```
plt.plot(x - width/2, student2, linestyle='--', marker='s',  
label="Student 2")
```

```
plt.title("Performance Comparison of Two Students")
```

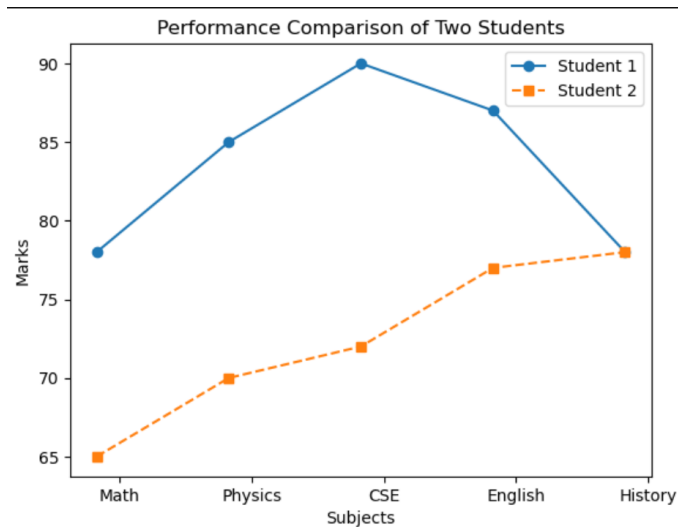
```
plt.xlabel("Subjects")
```

```
plt.ylabel("Marks")
```

```
plt.xticks(x, subjects)
```

```
plt.legend()
```

```
plt.show()
```



Output:

Student 1 consistently outperforms Student 2 in all subjects. The largest gap appears in CSE (90 vs 72), indicating stronger computing skills. Student 2 shows moderate but consistent performance. Overall, student 1 performs better than student 2.

Task 3: Bar Chart

Visualize student enrollment in departments. Display values above bars and discuss which department has the highest enrollment.

```
departments = ["CSE", "EEE", "BBA", "Civil", "English"]
```

```
students = [250, 180, 300, 120, 150]
```

Requirements:

- Add different colors.
- Display values on top of each bar.

```
departments = ["CSE", "EEE", "ME", "Civil", "IPE"]
```

```
students = [200, 180, 300, 160, 150]
```

```
colors = ["blue", "green", "orange", "red", "purple"]
```

```
bars = plt.bar(departments, students, color=colors)
```

```
for bar in bars:
```

```
    height = bar.get_height()
```

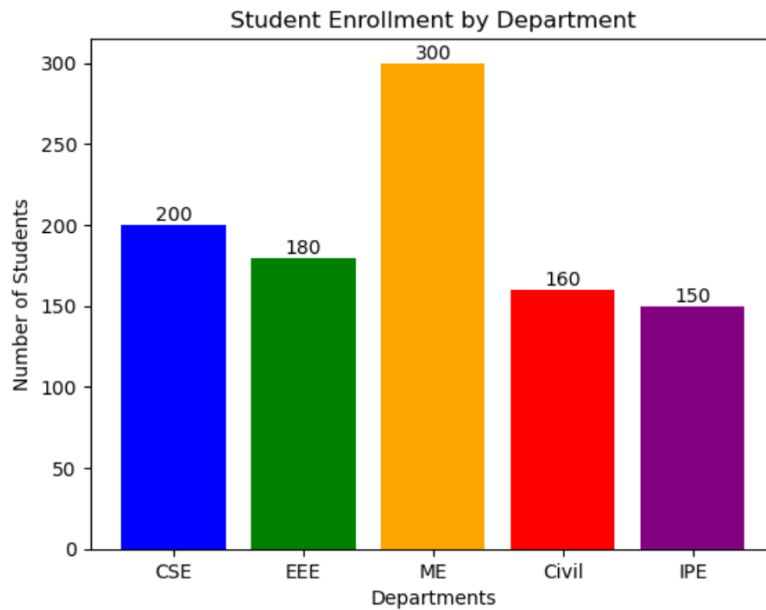
```
plt.text(bar.get_x() + bar.get_width()/2, height, str(height),
ha='center', va='bottom')
```

```
plt.title("Student Enrollment by Department")
```

```
plt.xlabel("Departments")
```

```
plt.ylabel("Number of Students")
```

```
plt.show()
```



Output:

As we can see, ME has the highest enrolment.

Task 4: Histogram

Plot marks distribution. Choose appropriate bins and explain whether the data is symmetric, skewed, or clustered.

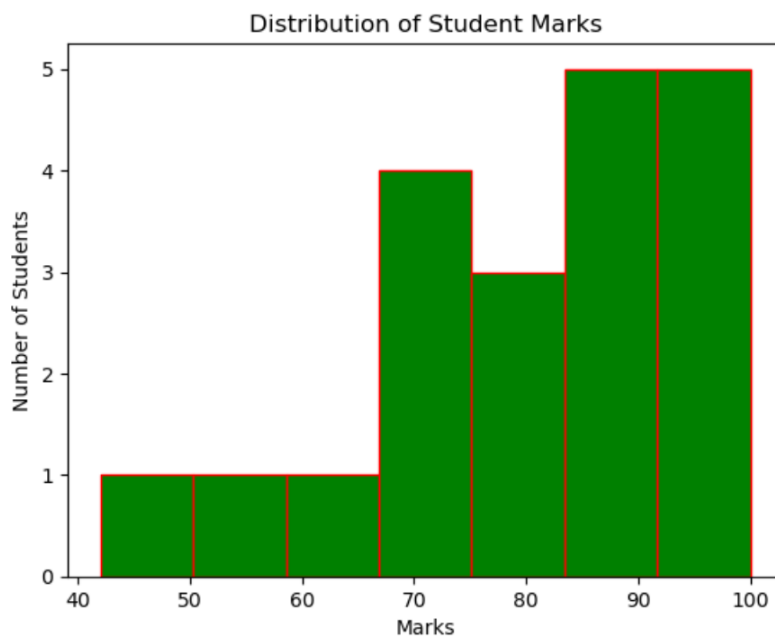
marks = [65, 70, 75, 80, 85, 90, 72, 68, 95, 88,
77, 83, 91, 73, 69, 81, 86, 92, 78, 74]

Requirements:

- Use appropriate bins.
- Add title and axis labels.
- Explain the distribution of marks.

```
marks = [42, 55, 60, 67, 71, 73, 75, 78, 80, 82,
         84, 85, 87, 89, 90, 92, 94, 96, 98, 100]

plt.hist(marks, bins=7, color= "green", edgecolor="red")
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.show()
```



Output:

The distribution is slightly left-skewed (negatively skewed) because most students scored high marks while a few students scored much lower. The data is clustered in the higher mark ranges, indicating generally good academic performance.

Task 5: Pie Chart

Show daily activity distribution. Display percentages and highlight the largest segment.

```
activities = ["Study", "Sleep", "Exercise", "Entertainment", "Others"]
```

```
hours = [6, 8, 1, 4, 5]
```

Requirements:

- Display percentages.
- Highlight the largest segment.
- Add a title.

```

activities = ["Study", "Sleep", "Exercise", "Entertainment",
"Others"]

hours = [6, 8, 1, 4, 5]

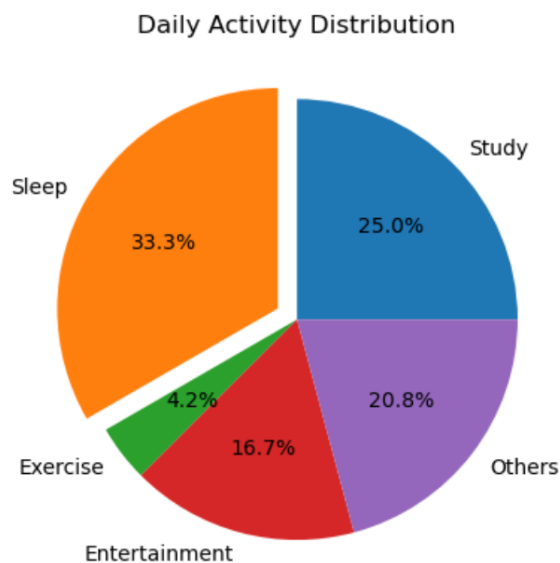
explode = [0, 0.1, 0, 0, 0]

plt.pie(hours, labels=activities, autopct='%1.1f%%',
explode=explode)

plt.title("Daily Activity Distribution")

plt.show()

```



Output:

Task 6: Student Performance Dashboard

Task: Create a figure containing four subplots using Matplotlib to visualize different aspects of student performance.

Requirements

Create a dashboard with the following plots:

1. Line Plot: Monthly Attendance

- Display attendance percentage for six months.
- Include title, axis labels, markers, and grid.

2. Bar Chart: Subject-wise Marks

- Display marks obtained in different subjects.
- Include title and axis labels.
- Show marks on top of each bar.

3. Pie Chart: Daily Time Allocation

- Show how a student spends time in different activities.
- Display percentage values.
- Highlight the largest section.

4. Histogram: Exam Score Distribution

- Show the frequency distribution of exam scores.
- Include appropriate bins, title, and axis labels.

Additional Requirements

- Use `plt.subplots(2, 2)` to create the dashboard.
- Set a suitable figure size.
- Add titles to all subplots.
- Use `plt.tight_layout()` to avoid overlapping elements.
- Save the dashboard as `student_dashboard.png`.
- Include comments in your code.

```
# Monthly Attendance
months = ["Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
attendance = [78, 82, 85, 88, 91, 93]

# Subject-wise Marks
subjects = ["CSE", "Math", "Statistics", "DBMS", "English"]
marks = [92, 85, 78, 88, 81]

# Daily Activities (Hours)
activities = ["Classes", "Sleep", "Coding", "Social Media", "Others"]
hours = [5, 7, 4, 2, 6]

# Exam Scores
scores = [52, 58, 61, 65, 67, 69, 72, 74, 76, 78,
          81, 83, 85, 87, 89, 91, 93, 95, 97, 99]
fig, ax = plt.subplots(2, 2, figsize=(12, 8))
```

```
# Line Plot: Monthly Attendance
ax[0, 0].plot(months, attendance, marker='o')
ax[0, 0].set_title("Monthly Attendance")
ax[0, 0].set_xlabel("Month")
ax[0, 0].set_ylabel("Attendance (%)")
ax[0, 0].grid(True)

# Bar Chart: Subject-wise Marks
bars = ax[0, 1].bar(subjects, marks, color='orange')
ax[0, 1].set_title("Subject-wise Marks")
ax[0, 1].set_xlabel("Subjects")
ax[0, 1].set_ylabel("Marks")

for bar in bars:
    ax[0, 1].text(bar.get_x() + bar.get_width()/2, bar.get_height(),
str(bar.get_height()), ha='center')

# Pie Chart: Daily Time Allocation
explode = [0, 0.1, 0, 0, 0]

ax[1, 0].pie(hours, labels=activities, autopct='%1.1f%%',
explode=explode)
ax[1, 0].set_title("Daily Time Allocation")

# Histogram: Exam Score Distribution
ax[1, 1].hist(scores, bins=7, color="violet", edgecolor="blue")
ax[1, 1].set_title("Exam Score Distribution")
ax[1, 1].set_xlabel("Score")
ax[1, 1].set_ylabel("Frequency")

plt.tight_layout()
plt.savefig("student_dashboard.png")
plt.show()
```

Output:

